

Semi Online

Divyanshu, Utsav

May 8, 2024

Contents

1	Motivation	2
2	Introduction	2
2.1	IID setting with multiple queries	2
2.2	Non IID setting with single query	3
2.3	Competitive Ratio	3
3	Past Works	3
4	IID Setting with Multiple Queries: Correction in Hoefer Etal.'s work	4
5	Non-IID Setting	4
5.1	Formal statement	4
5.2	Dynamic Programming Algorithm	4
5.3	Decision Tree	5
5.4	Optimal strategy for the last random variable	5
5.5	Optimal strategy for the second last random variable	6
5.5.1	Finding the optimal threshold	7
5.5.2	Analysing the reward function	7
5.6	Constructing the Dual Problem	8
5.7	Worst Case Analysis: Uniform Distribution	10
5.8	Worst Case Analysis: 3-point Distribution	10
6	Appendix	12
6.1	IID with Multiple Queries	12
6.1.1	Proving that the stated algorithm's competitive ratio is not $1 - o(1)$	13
6.1.2	Suggesting a simple fix	15
6.2	Dynamic Programming Algorithm Explanation	15
6.3	Worst case for $n=2$ and uniform distributions	15
6.4	Worst case for $n=2$ and 3 point distribution	16

1 Motivation

Consider an application process in which n job candidates are interviewed sequentially one by one for a single position. For each candidate, we assume the qualification for the job can be expressed by an i.i.d. non-negative random variable X_i with known distribution F . The goal is to maximize the expected value of the selected candidate. To which extent is the optimal achievable value harmed by the online arrival of the candidates? This is the classic gambler's problem or the *prophet problem*. This is what we call an **online** prophet problem, since the decision maker has to immediately say Yes or No to the candidates.

The prophet problem has been extremely popular over the last decades, but the assumption that an interview can reveal the full realization of the random variable seems unrealistic. Even after a long interview, an interviewer is usually not fully aware of the entire set of exact qualifications of a candidate. And it is often not the case that we need immediately take a decision, a lot of real world settings allow us to pick candidates after interviewing all of them. Lastly assuming that all candidates arrive from the same distribution is also not usually true. Candidates with different age, experience, education will often tend to come from different distributions. We examine the consequences of an arguably more realistic set of conditions. First, we assume that each candidate only partly reveals their realization in the form of a Yes/No question. Second, we assume that the selection must be made only at the end of the process. This is referred to as the **semi-online prophet problem**.

2 Introduction

We work on the semi-online prophet problem in our project. It is a variation of the classic prophet problem. The candidates are random variables from distributions that are known to us beforehand. We measure the performance of our algorithm by comparing the Expected payoff of our strategy against that of a "prophet", hence the name. The "prophet" already knows all the realisations of the random variables, hence it picks the maximum valued candidate. The semi online setting works as follows, we know the distributions of all the random variables, we are allowed to do queries to each of the random variable, which are Yes/No questions. Based on the information we obtain from our queries we decide to pick the element with the highest conditional expectation. There can be a lot of variations to this depending on the following factors :

- The kind of query that you are allowed to make.
- If the random variables involved are iid or not.
- The number of queries that you are allowed to make to each random variable and the total number of queries.

Over here we work on two of the scenarios. We provide the formal description of each below :

2.1 IID setting with multiple queries

We are provided with n iid random variables $X_1, X_2, \dots, X_n$, we know their distribution F . We can ask queries to an item, such that we restrict ourselves to a total of n queries. We may ask multiple queries about a given item. Each query is a Yes/No answer to a threshold τ / Yes indicating that

the realisation of the item is above the threshold. We are allowed to make a total of n queries possibly doing multiple queries. The queries are non-adaptive, ie. depending on the response of the previous item, we can decide the query for the next item. Our goal is to maximise the value of the selected item.

2.2 Non IID setting with single query

We are provided with n random variables $X_1, X_2, \dots, X_n$ and each of them has a different distribution that is known to us. We can ask exactly one query to each item. Each query is a Yes/No answer to a threshold τ / Yes indicating that the realisation of the item is above the threshold. Thresholds are allowed to be adaptive.

2.3 Competitive Ratio

In both of the cases we measure the goodness of our algorithm using a quantity known as the competitive ratio which is defined as follows :

$$\text{Competitive Ratio} = \frac{\mathbb{E}[ALG]}{\mathbb{E}[OPT]} \quad (1)$$

The expectation is over the random variables $X_1, X_2, \dots, X_n$ and over the algorithm (if it is randomised). Our goal is to come up with an algorithm that maximizes the competitive ratio, over the worst case n and F .

3 Past Works

Thus problem has been analysed in similar settings in the past works by Hoefer Etal. [2] and Harb Elfarouk [3].

In particular Hoefer Etal. showed the following results :

- The optimal non-adaptive testing algorithm for boxes obtains at most the expected reward of the optimal algorithm for the gambler's problem with $n+1$ boxes.
- For finite discrete distributions, an optimal testing algorithm can be computed by dynamic programming in polynomial time.
- There exists no $(1 - o(1))$ competitive algorithm for threshold testing
- There is an efficient $(1 - o(1))$ competitive algorithm for threshold testing with multiple test per box and a continuous iid distribution
- They provide an efficient $0.869 - o(1)$ competitive algorithm for iid threshold testing with a continuous distribution. This result however was immediately improved by Elfarouk Harb

The 0.869 algorithm essentially worked by computing 3 quantile thresholds with increasing value. The random variables are queried on the smallest threshold, and whenever we obtain a positive test, we update our threshold to a higher value. Finally we pick the element with the highest conditional

expectation.

Following up on [2] and incorporating his ideas of sharding that improve computation in discrete scenarios Elfarouk Harb in his [3] beats the $0.869 - o(1)$ result. The main contributions to this domain from [3] are :

- There exists an algorithm for the IID semi online problem that has a competitive ratio of atleast 0.89
- There is an algorithm for the semi online prophet problem with multiple queries that is $1 - o(1)$ competitive for both the iid and the non iid setting.
- An upper bound of ≈ 0.92 for the semi online iid prophet problem, that he claims to know from a private correspondence with authors of [2].

4 IID Setting with Multiple Queries: Correction in Hoefer Etal.'s work

We reviewed the following paper [2] by Hoefer Etal.. They provide with an algorithm that claims a $1 - o(1)$ competitive ratio for the semi online problem on iid variables with multiple queries allowed. We reviewed their calculatoins and found a flaw in them. We had the following findings :

- The algorithm provided in [2] does not guarantee a $1 - o(1)$ competitive ratio.
- The payoff of their algorithm is upper bounded by $1 - \frac{1}{2e}$
- We provide a change in the working of their algorithm that guarantees a $1 - o(1)$ competitive ratio

Look at the appendix 6.1 for the relevant details.

5 Non-IID Setting

5.1 Formal statement

You are given n random variables $X_n, X_{n-1}, \dots, X_1$, along with thier distributions. Your goal is to pick the maximum valued element in expectation. You can make a single Yes/No query to each variables, where you ask a threshold, and Yes indicates that the realisation is above the threshold, while, No indicates that the realisation is below the threshold. Your aim is to maximise the competitive ratio of your scheme.

5.2 Dynamic Programming Algorithm

We came up with a dynamic programming representation of the optimal payoff strategy.



For X_k , suppose that from your previous queries on $X_n, \dots, X_{k+1}$, suppose the highest conditional expectation that you have obtained is given by μ . Then the Algorithm's payoff and the choice of optimal threshold to query for X_k can be expressed as the expression below :

$$ALG_k(\mu) = \max_{\tau} (F_k(\tau)ALG_{k-1}(\max(\mu, \mathbb{E}[X_k|X_k < \tau])) + (1-F_k(\tau))ALG_{k-1}(\max(\mu, \mathbb{E}[X_k|X_k \geq \tau])))$$

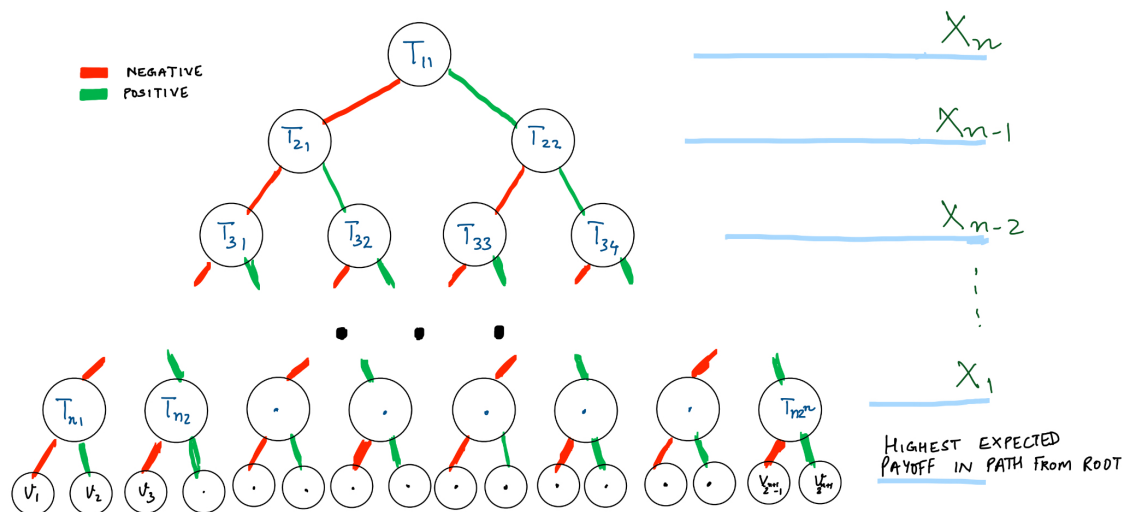
Look at the appendix 6.2 to understand how we reach this expression.

The τ that maximises the above expression is the optimal threshold.

The payoff of the DP algorithm is given by $ALG_n(0)$.

5.3 Decision Tree

We observe that given an instance of n random variables $X_n, \dots, X_2, X_1$ we can represent the optimal algorithm as a decision tree of $n+1$ levels. Each level in the tree, except for the leaves, corresponds to a particular random variable. The first level is for X_n , the second last for X_1 and the leaves indicates the choice we make. Each node has a threshold which is the optimal choice to be asked to a random variable that has been reached through that path from the root. Based on the response Yes or No the query we branch of to the right or the left child respectively. The leaves correspond to the element that the algorithm that has followed the above query path from the root decides to pick. We will further see that this representation of the algorithm facilitates in constructing the LP of the problem.



5.4 Optimal strategy for the last random variable

Now we are looking at the optimal threshold for the last Random Variable.

Lemma: The optimal threshold for the last variable is μ where μ is the highest conditional expectation till now (i.e. before last variable).

Proof :

Let the threshold be t and F be the distribution of the last random variable

Expected Reward: $F(t) \max(\mathbb{E}[X_1 | X_1 < t], \mu) + (1 - F(t)) \max(\mathbb{E}[X_1 | X_1 \geq t], \mu)$

There are two possible cases. $\mu < \mathbb{E}[X_1]$ or $\mu > \mathbb{E}[X_1]$

Case 1: $\mu < \mathbb{E}[X_1]$

For this case we have $\mu < \mathbb{E}[X_1] \leq \mathbb{E}[X_1 | X_1 \geq t]$

If $\mu < \mathbb{E}[X_1 | X_1 < t]$ then the payoff is simply $\mathbb{E}[X_1]$

Otherwise we have,

$$\mathbb{E}[\text{Payoff}(t)] = F(t) \cdot \mu + (1 - F(t)) \cdot \mathbb{E}[X_1 | X_1 \geq t]$$

Taking derivative with respect to t

$$\begin{aligned} \frac{d\mathbb{E}[\text{Payoff}(t)]}{dt} &= \frac{dF(t) \cdot \mu}{dt} + \frac{d(1 - F(t)) \cdot \mathbb{E}[X_1 | X_1 \geq t]}{dt} \\ &= \mu \cdot f(t) - f(t) \cdot t \end{aligned}$$

Hence we obtain a single extrema, and this corresponds to a maxima at $t = \mu$

Also $\mu > \mathbb{E}[X_1 | X_1 < \mu]$ which satisfies the condition

Case 2: $\mu \geq \mathbb{E}[X_1]$

For this case we have $\mu \geq \mathbb{E}[X_1] \geq \mathbb{E}[X_1 | X_1 < t]$

If $\mu > \mathbb{E}[X_1 | X_1 \geq t]$ then the payoff is simply μ

Otherwise we have,

$$\mathbb{E}[\text{Payoff}(t)] = F(t) \cdot \mu + (1 - F(t)) \cdot \mathbb{E}[X_1 | X_1 \geq t]$$

Taking derivative with respect to t

$$\begin{aligned} \frac{d\mathbb{E}[\text{Payoff}(t)]}{dt} &= \frac{dF(t) \cdot \mu}{dt} + \frac{d(1 - F(t)) \cdot \mathbb{E}[X_1 | X_1 \geq t]}{dt} \\ &= \mu \cdot f(t) - f(t) \cdot t \end{aligned}$$

Hence we obtain a single extrema, and this corresponds to a maxima, at $t = \mu$

Also $\mu < \mathbb{E}[X_1 | X_1 \geq \mu]$ which satisfies the condition

For both cases we obtain the same expression which gives us the result

$$\mathbb{E}[\text{Payoff}(t)] = \mathbb{E}[\max(X_1, \mu)]$$

5.5 Optimal strategy for the second last random variable

After getting the optimal threshold for the last Random Variable we tried working with the second last Random Variable.

Key terms:

1. X_2 : Second last Random Variable

2. X_1 : Last Random Variable
3. F_2 : Distribution of second last Random Variable
4. F_1 : Distribution of last Random Variable
5. t : Threshold for second last Random Variable
6. μ : Expected highest reward before querying X_2
7. $\mu_1(t)$: $\mathbb{E}[X_2|X_2 < t]$
8. $\mu_2(t)$: $\mathbb{E}[X_2|X_2 \geq t]$

$$\mathbb{E}[\text{Payoff}(t)] = F_2(t) \cdot \mathbb{E}[\max(X_1, \max(\mu_1, \mu))] + (1 - F_2(t)) \cdot \mathbb{E}[\max(X_1, \max(\mu_2, \mu))]$$

First we tried working with the setting where there are only two random variables or in other words $\mu = 0$

5.5.1 Finding the optimal threshold

First we tried simply differentiating the expression and getting t as some function of F_1 and F_2

$$\mathbb{E}[\text{Payoff}(t)] = F_2(t) \cdot \mathbb{E}[\max(X_1, \mu_1)] + (1 - F_2(t)) \cdot \mathbb{E}[\max(X_1, \mu_2)]$$

Taking derivative with respect to t

$$\begin{aligned} \frac{d\mathbb{E}[\text{Payoff}(t)]}{dt} &= \frac{dF_2(t) \cdot \mathbb{E}[\max(X_1, \mu_1)]}{dt} + \frac{d(1 - F_2(t)) \cdot \mathbb{E}[\max(X_1, \mu_2)]}{dt} \\ &= f_2(t) \cdot (F_1(\mu_2) \cdot \mu_2 + \mathbb{E}[X_1 \cdot \mathbb{I}[X_1 \geq \mu_2]] - F_1 \cdot \mu_1 - \mathbb{E}[X_1 \cdot \mathbb{I}[X_1 \geq \mu_1]] + \mu_1 \cdot f_1(\mu_1) \cdot (\mu_1 - t)) \end{aligned}$$

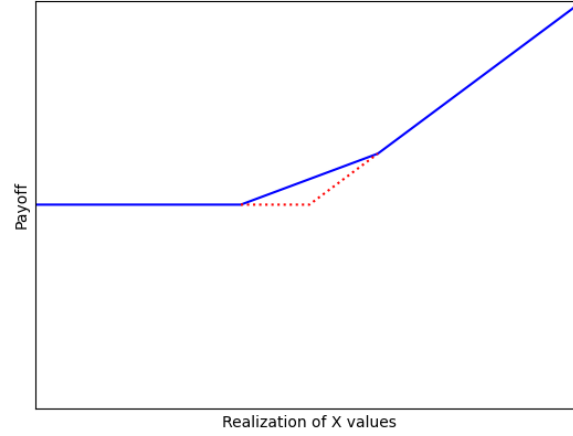
We were not able to simplify this expression anymore to get a expression for optimal t

5.5.2 Analysing the reward function

Next approach we tried was dividing the range of X_1 into three regions to get a better idea about $\mathbb{E}[\text{Payoff}(t)]$

1. $X_1 < \mu_1$
Payoff will be simply $\mathbb{E}[X_2]$
2. $\mu_1 \leq X_1 \leq \mu_2$
Payoff will be $F_2(t) \cdot X_1 + (1 - F_2(t)) \cdot \mu_2$
3. $X_1 \geq \mu_2$
Payoff will be X_1

If we plot value of X_1 vs Payoff it will look like this:



From the graph we can clearly see that every value of t achieves at least the reward that is below the dotted lines

The triangle is the extra area which will depend on the value of t and it will be

$$\int_{x=\mu_1}^{\mathbb{E}[X_2]} F_2(t) \cdot (x - \mu_1) \cdot f_1(x) \cdot dx + \int_{x=\mathbb{E}[X_2]}^{\mu_2} (1 - F_2(t)) \cdot (\mu_2 - x) \cdot f_1(x) \cdot dx$$

We tried finding the optimal value of t for which the area of this triangle is the largest by observing the properties of this area function.

The derivative of the payoff function is increasing in the interval μ_1 to μ_2 otherwise 0

Using this we tried to get some convexity or concavity condition so as to predict optimal t but were not able to get such a condition.

5.6 Constructing the Dual Problem

In the past works such as [1] and [4] where it was observed that the ex-ante prophet problem is the **dual** of the Online Contention Resolution Scheme problems. This greatly simplified finding optimal bounds for the latter. Inspired from this we try to find the dual of our problem and then we will try to map it with a CRS.

We work with **3-point** distributions. This is because this is the simplest non trivial scenario, since for 2 point distributions we can simply get to know the realisation of all the elements and pick the max. Without loss of generality we assume that X_i has the following distribution :

$$F_i = \begin{cases} 0 & \text{w.p.} & 1 - p_i - q_i \\ a_i & \text{w.p.} & p_i \\ a_i + b_i & \text{w.p.} & q_i \end{cases}$$

Where $a_i, b_i \geq 0$ For three point distributions note that the choice of thresholds is not continuous rather it is discrete. All we have to decide is that whether the threshold has to **low** : present

between 0 and a_i or **high**, present between a_i and $a_i b_i$. Hence each node in the decision tree 5.3, can possibly take only 2 values. Every assignemnt on high or low at the non leaf nodes corresponds to a new algorithm. Hence we see that there are total of $M = 2^{2^n-1} \cdot n^{2^n}$ possible. Let us call them $A_1, A_2, \dots, A_m$. These are finitely many algorithms.

We define the following four quantities :

- p_{ij} : Probability that A_i picks the j th element, where X_j was asked a high threshold and it gave a positive response. The conditional expectation of X_j in this sceanrio is : $a_i + b_i$
- q_{ij} : Probability that A_i picks the j th element, where X_j was asked a high threshold and it gave a neagative response.. In this sceanrio the conditional expectation is $\frac{p_i a_i}{1 - q_i}$
- r_{ij} : Probablity that A_i picks the j th element, where X_j was asked a low threshold and it gave a positive response. The conditional expectation in this scenario is $\frac{p_i a_i + (a_i + b_i) q_i}{p_i + q_i}$

We can write the Payoff of

$$\text{Payoff } A_i = \sum_{j=1}^n \left((a_i + b_i) p_{ij} + q_{ij} \frac{(p_i a_i)}{1 - q_i} + r_{ij} \frac{p_i a_i + (a_i + b_i) q_i}{p_i + q_i} \right)$$

Further more to construct the lp we add two more restrictions. These are as follows :

1. $\sum_{i=1}^n (p_i + q_i) \leq 1$
2. $\sum_{i=1}^n (a_i p_i + (a_i + b_i) q_i) = 1$

The above two conditions guarantee that payoff of the ex-ante prophet is 1. Since the first condition says that ex-ante can always pick whenever an element shows up with non zero value, and the second tell the payoff in such a scenario. The competitive ratio now is just the value obtained by the algorithm in the worst case. Let us denote it by d . Then we obtain the following minimisation problem :

Minimise d , such that :

$$d \geq \sum_{i=1}^n \left((a_i + b_i) p_{ji} + q_{ji} \frac{(p_i a_i)}{1 - q_i} + r_{ji} \frac{p_i a_i + (a_i + b_i) q_i}{p_i + q_i} \right) \quad \forall j \quad (3)$$

$$1 \geq \sum_{i=1}^n (p_i + q_i) \quad (4)$$

$$1 = \sum_{i=1}^n (a_i p_i + (a_i + b_i) q_i) \quad (5)$$

$$a_i \geq 0 \quad \forall i \quad (6)$$

$$b_i \geq 0 \quad \forall i \quad (7)$$

The dual of the above system is the following :

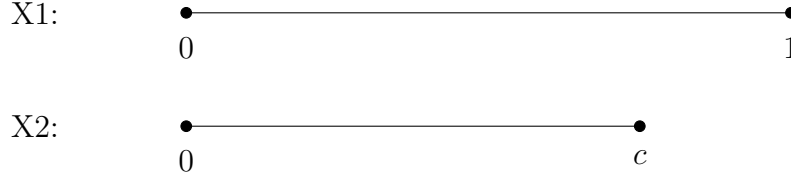
$$\begin{aligned}
& \text{Maximise } x \text{ subject to :} & (8) \\
& \sum_{j=1}^m \lambda_j \left(p_{ji} + \frac{q_{ji}p_i}{1-q_i} + r_{ji} \right) \geq x(p_i + q_i) & \forall i = 1, \dots, n \\
& \sum_{j=1}^m \lambda_j \left(p_{ji} + \frac{r_{ji}q_i}{p_i + q_i} \right) \geq x(q_i) & \forall i = 1, \dots, n \\
& \lambda_j \geq 0 & \forall j = 1, \dots, m & (9)
\end{aligned}$$

We were however unable to interpret these dual conditions as Contention Resolution problem or something similar.

5.7 Worst Case Analysis: Uniform Distribution

Now we tried to get an upper bound on the competitive ratio of the optimal algorithm and the first case we looked at was uniform random variables.

Here we have two random variables both uniformly distributed



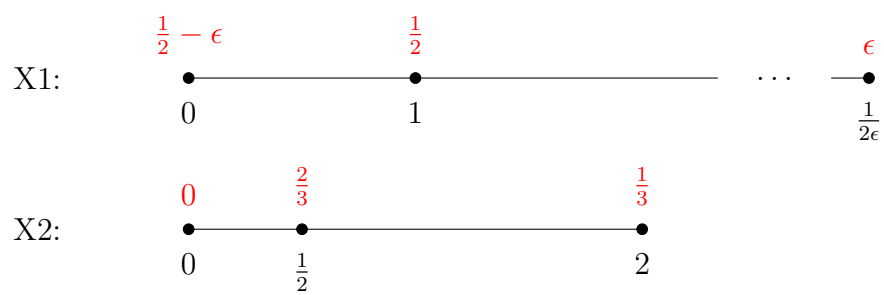
By analysing for different possible values of c we got that the worst case is obtained for $c = 0.5$ and competitive ratio for this case will be $\frac{56}{57} \approx 0.9824$

This is worse than the known competitive ratio for the iid setting and clearly non-iid settings are harder than the iid setting, so we shifted to other types of distributions.

5.8 Worst Case Analysis: 3-point Distribution

We try to obtain the worst-case distribution on 2 random variables where the distributions are supported on 3 discrete points. The motivation behind this is that for any value of n and 2 point distributions we can obtain a $CR = 1$. Hence 3 point distribution and $n=2$ is in a sense the simplest case.

We obtain a concrete example that admits a $0.9 + \epsilon$ competitive ratio, hence we obtain an upper bound of 0.9. **This is strictly better than what is claimed by Elfarouk Harb in [3] 3.** The case looks as follows :



The values in red indicate the probability mass on each point.

6 Appendix

6.1 IID with Multiple Queries

The algorithm provided by [2] is as follows :

In the first step discard the last $\lceil n^{2/3} \rceil$ boxes. The remaining ones are tested for the threshold $F^{-1}(1 - n^{-2/3})$. Let P be the set of items that were tested positively. For each item $i \in P$, the algorithm next searches the integers $0, \dots, \lceil n^{1/3} \rceil$ to find the largest j from this set such that the test for $F^{-1}(1 - n^{-2/3} + j/n)$ is positive. Using binary search this requires $O(\log(n))$ tests for each item $i \in P$. Using the result, we can identify that item is of type j . The algorithm may well run out of tests, but if it succeeds on determining the type of each item, then it picks the item that is present in the highest type. If no such item exists, it is not unique, or the algorithm ran out of tests before determining the type of each item, it may choose an arbitrary item and terminate. To analyse, we fix any $v \in [F^{-1}(1 - n^{-2/3}), F^{-1}(1)]$. We denote by $\mathcal{M}_v$ the event that $\max X_1, \dots, X_n = v$, and by X_σ the value obtained by the algorithm. Our goal is to show that, whenever an optimal box has a value v , the probability that we chose an optimal box is :

$$\mathbb{P}[X_\sigma = v | \mathcal{M}_v] = 1 - o(1)$$

Now we see an optimal box with value v with probability :

$$\mathbb{P}[\max X_1, X_2, \dots, X_n \geq F^{-1}(1 - n^{-2/3})] = 1 - (1 - n^{-2/3})^n = 1 - o(1)$$

We define the following events :

$\mathcal{E}_1$ is the event that the algorithm does not run out of test for large-enough n

$\mathcal{E}_2$ is the event that only a single box has the largest type.

Further that note that $\mathbb{P}[X_\sigma = v | \mathcal{M}_v] \geq \mathbb{P}[\mathcal{E}_1 \cap \mathcal{E}_2 | \mathcal{M}_v]$ since our algorithm chooses the box with the maximum value if $\mathcal{M}_v, \mathcal{E}_1$ and $\mathcal{E}_2$ occur.

$$\begin{aligned} \mathbb{P}[\mathcal{E}_1 \cap \mathcal{E}_2 | \mathcal{M}_v] &= \mathbb{P}[\mathcal{E}_1 | \mathcal{M}_v] \cdot \mathbb{P}[\mathcal{E}_2 | \mathcal{M}_v \cap \mathcal{E}_1] \\ &\geq (1 - e^{-\frac{n^{1/3}}{3}}) \cdot (1 - \frac{n^{-1}}{1 - n^{-2/3}})^{n^{1/2}} = 1 - o(1) \end{aligned}$$

But this is where we found a discrepancy on how authors calculate $\mathbb{P}[\mathcal{E}_2 | \mathcal{M}_v \cap \mathcal{E}_1]$. The lower bound of the probability that an element does not fall in the bucket of the maximum element is given by $(1 - \frac{n^{-1}}{1 - n^{-2/3}})$ only if we have not conditioned the element to be in the top $n^{-2/3}$ quantile. As soon as we condition it to be in the top $n^{-2/3}$ this probability changes. Hence using the above expression with the exponent of $n^{1/2}$ is incorrect as it evaluates it only for those elements that are known to be in the top $n^{-2/3}$ quantile.

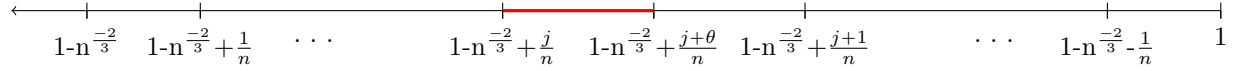
If $(1 - \frac{n^{-1}}{1 - n^{-2/3}})$, is raised to the power n , then this is not $1 - o(1)$.

$$\begin{aligned}
\mathbb{P}[\mathcal{E}_2 | \mathcal{M}_v \cap \mathcal{E}_1] &\geq \left(1 - \frac{n^{-1}}{1 - n^{-2/3}}\right)^n \\
&\approx \exp\left(\frac{-n^{-1}}{1 - n^{-2/3}} \cdot n\right) \\
&= \exp\left(\frac{-1}{1 - n^{-2/3}}\right) \\
&\approx \exp(-1) \\
&\approx 0.368
\end{aligned} \tag{10}$$

6.1.1 Proving that the stated algorithm's competitive ratio is not $1 - o(1)$

$$\begin{aligned}
\mathbb{P}[X_\sigma = v | \mathcal{M}_v] &= \mathbb{P}[\text{Only one in the max box} | \mathcal{M}_v] + \mathbb{P}[\text{More than one in the max box and max is picked} | \mathcal{M}_v] \\
&\leq \mathbb{P}[\mathcal{E}_2 | \mathcal{M}_v] + \frac{1}{2}(1 - \mathbb{P}[\mathcal{E}_2 | \mathcal{M}_v]) \\
&\leq \frac{1}{2} + \frac{\mathbb{P}[\mathcal{E}_2 | \mathcal{M}_v]}{2}
\end{aligned}$$

$$v = F^{-1}\left(1 - n^{-\frac{2}{3}} + \frac{j + \theta}{n}\right)$$



$$\begin{aligned}
\mathbb{P}[\mathcal{E}_2 | \mathcal{M}_v] &= \left(\frac{1 - n^{-\frac{2}{3}} + \frac{j}{n}}{1 - n^{-\frac{2}{3}} + \frac{j+\theta}{n}}\right)^{n-1} \\
&= \left(1 - \frac{\theta}{n - n^{\frac{1}{3}} + j + \theta}\right)^{n-1} \\
&\leq \left(1 - \frac{\theta}{n}\right)^{n-1} \\
&\leq (e^{-1})^{\frac{n-1}{n}\theta} \\
&\approx e^{-\theta}
\end{aligned}$$

Now,

$$\begin{aligned}
\mathbb{P}[X_\sigma = v | \mathcal{M}_v] &\leq \frac{1}{2} + \frac{\mathbb{P}[\mathcal{E}_2 | \mathcal{M}_v]}{2} \\
&\leq \frac{1}{2} + \frac{1}{2e^\theta}
\end{aligned}$$

$$\mathbb{P}[\text{Success of the algorithm}] = \int_{t=1-n\frac{-2}{3}}^1 \mathbb{P}[X_\sigma = F^{-1}(t)|\mathcal{M}_{F^{-1}(t)}] \cdot \mathbb{P}[\mathcal{M}_{F^{-1}(t)}] dt$$

$\mathbb{P}[\mathcal{M}_{F^{-1}(t)}]$ is the pdf of the max we denote it as (t) from now on. Note that (t) is an increasing function in t

Dividing into intervals

$$\begin{aligned} \mathbb{P}[\text{Success of the algorithm}] &= \sum_{j=0}^{n\frac{1}{3}-1} \int_{t=1-n\frac{-2}{3}+\frac{j}{n}}^{1-n\frac{-2}{3}+\frac{j+1}{n}} \mathbb{P}[X_\sigma = F^{-1}(t)|\mathcal{M}_{F^{-1}(t)}] \cdot (t) dt \\ &\leq \sum_{j=0}^{n\frac{1}{3}-1} \int_{t=1-n\frac{-2}{3}+\frac{j}{n}}^{1-n\frac{-2}{3}+\frac{j+1}{n}} \left(\frac{1}{2} + \frac{1}{2e^{n(t-(1-n\frac{-2}{3}+\frac{j}{n}))}} \right) \cdot (t) dt \end{aligned}$$

Here (t) is an increasing function and $\left(\frac{1}{2} + \frac{1}{2e^{n(t-(1-n\frac{-2}{3}+\frac{j}{n}))}} \right)$ is a decreasing function

Using Chebyshev's sum inequality : If $f(x)$ is increasing in (a, b) and $g(x)$ is decreasing in (a, b) then :

$$\begin{aligned} \frac{1}{b-a} \int_{x=a}^b f(x) \cdot g(x) dx &\leq \int_{x=a}^b \frac{1}{b-a} \cdot f(x) \cdot \int_{x=a}^b \frac{1}{b-a} \cdot g(x) \\ \mathbb{P}[\text{Success of the algorithm}] &\leq n \cdot \sum_{j=0}^{n\frac{1}{3}-1} \left(\int_{t=1-n\frac{-2}{3}+\frac{j}{n}}^{1-n\frac{-2}{3}+\frac{j+1}{n}} \left(\frac{1}{2} + \frac{1}{2e^{n(t-(1-n\frac{-2}{3}+\frac{j}{n}))}} \right) dt \cdot \int_{t=1-n\frac{-2}{3}+\frac{j}{n}}^{1-n\frac{-2}{3}+\frac{j+1}{n}} (t) dt \right) \\ &= n \cdot \int_{t'=0}^{\frac{1}{n}} \left(\frac{1}{2} + \frac{1}{2e^{nt'}} \right) dt' \cdot \int_{t=1-n\frac{-2}{3}}^1 (t) dt \\ &\leq n \cdot \int_{t'=0}^{\frac{1}{n}} \left(\frac{1}{2} + \frac{1}{2e^{nt'}} \right) dt' \\ &= \frac{1}{2} + \frac{1}{2} (1 - e^{-1}) \\ &= 1 - \frac{1}{2e} \end{aligned}$$

6.1.2 Suggesting a simple fix

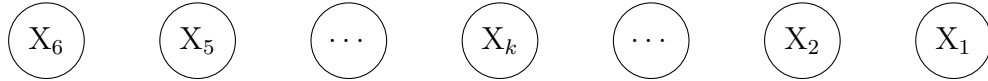
We suggest a slight correction in the algorithm that guarantees a $1 - o(1)$ payoff competitive ratio. **Divide the top $n^{-2/3}$ quantile into $n^{4/3}$ boxes instead of n .** This leads to each box being of the size $\frac{n^{-2/3}}{n^{4/3}} = \frac{1}{n^2}$. The proof presented by the authors now works. We use n^{-2} as an upper bound on the probability that an independently drawn value has any fixed type, and $1 - n^{-\frac{2}{3}}$ as a lower bound on the probability that such a value is below v .

$$\begin{aligned}\mathbb{P}[\mathcal{E}_2 | \mathcal{M}_v \cap \mathcal{E}_1] &\geq \left(1 - \frac{n^{-2}}{1 - n^{-2/3}}\right)^n \\ &= 1 - o(1)\end{aligned}$$

Rest of the calculation done by the authors remains the same, and we obtain that :

$$\mathbb{P}[X_\sigma = v | \mathcal{M}_v] \geq \mathbb{P}[\mathcal{E}_2 | \mathcal{M}_v \cap \mathcal{E}_1] = 1 - o(1) \quad (11)$$

6.2 Dynamic Programming Algorithm Explanation



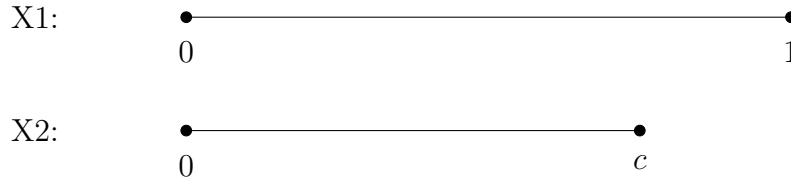
For X_k , suppose that from your previous queries on $X_n, \dots, X_{k+1}$, the highest conditional expectation that you have obtained is given by μ . Then we try to find the optimal threshold τ to query X_k .

Suppose we pick τ , then the probability that X_k is below τ is given by $F_k(\tau)$, in this case the condition expectation of X_k is $\mathbb{E}[X_k | X_k < \tau]$ and thus when we move on to the next step to query X_{k-1} , the highest conditional expectation observed would be $\max(\mu, \mathbb{E}[X_k | X_k < \tau])$ and hence the payoff of X_k in this scenario would be $ALG_{k-1}(\max(\mu, \mathbb{E}[X_k | X_k < \tau]))$.

Similarly the other scenario ie. when the test is positive happens with probability $(1 - F_k(\tau))$, the updated μ would be $\max(\mu, \mathbb{E}[X_k | X_k \geq \tau])$ and the payoff would be $ALG_{k-1}(\max(\mu, \mathbb{E}[X_k | X_k \geq \tau]))$. We thus pick the τ that maximises this weighted reward. And express $ALG_k(\mu)$ as the expected value of this reward using the optimal τ

$$ALG_k(\mu) = \max_{\tau} (F_k(\tau) ALG_{k-1}(\max(\mu, \mathbb{E}[X_k | X_k < \tau])) + (1 - F_k(\tau)) ALG_{k-1}(\max(\mu, \mathbb{E}[X_k | X_k \geq \tau])))$$

6.3 Worst case for $n=2$ and uniform distributions



Caes 1: $c \geq 1$

Let the threshold for the first random variable be t

$$\text{Payoff} = \frac{-t^2 + t + 4c^2 + 1}{8c}$$

$$\text{Optimal} = \frac{4c^2 + \frac{4}{3}}{8c}$$

Best case will be when $t = 0.5$

Minimum CR will be at $c = 1$

$$\text{CR} = \frac{63}{64}$$

Case 2: $0.75 \leq c \leq 1$

$$\text{Competitive Ratio} = \frac{4c^2 + \frac{5}{4}}{\frac{4c^3}{3} + 4c}$$

Minimum CR will be at $c = 0.75$

$$\text{CR} = \frac{56}{57}$$

Case 3: $0.5 \leq c \leq 0.75$

$$\text{Competitive Ratio} = \frac{\frac{32c^3}{27} + 4c}{\frac{4c^3}{3} + 4c}$$

Minimum CR will be at $c = 0.5$

$$\text{CR} = \frac{116}{117}$$

Case 3: $c \leq 0.5$

$$\text{Competitive Ratio} = \frac{\frac{32c^3}{27} + 4c}{\frac{4c^3}{3} + 4c}$$

Minimum CR will be at $c = 0.5$

$$\text{CR} = \frac{116}{117}$$

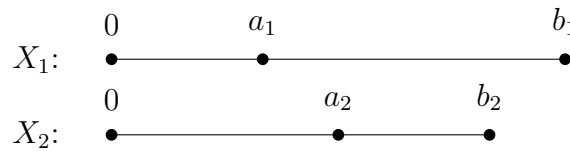
6.4 Worst case for n=2 and 3 point distribution

We systematically create the worst case instance here. Without loss of generality we fix one point of both the distributions at 0. And let the other two points be $a_1 < b_1$ and $a_2 < b_2$. Moreover denote the probability mass at these points by p_1, q_1 and p_2, q_2 respectively.

We call a threshold low if it is between 0 and a and high if it is between a and b .

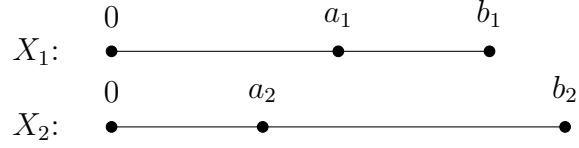
There are several cases that are clearly not the worst cases, since we can pick the maximum element by picking appropriate thresholds such as :

- $a_1 < a_2 < b_2 < b_1$:



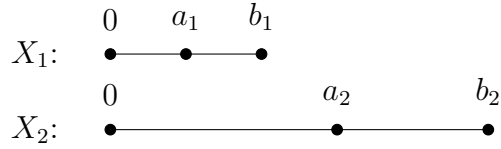
In this case you choose your first threshold to be **high**, if the response is positive, we simply pick b_1 the next query is not needed. Otherwise in case of a negative query we ask a **low** threshold if the response is positive, we pick X_2 else we pick X_1 .

- $a_2 < a_1 < b_1 < b_2$:



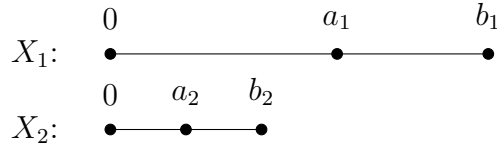
In this case you ask first query to be a **low** query, and second query to be a **high** query. If the response of second query is positive you simply pick X_2 , if it is negative, then you look at response of the first query, if it was positive pick X_1 else pick X_2

- $a_1 < b_1 < a_2 < b_2$:



You simply ask low thresholds to both the variables. If X_2 has a positive response pick X_2 else pick X_1 .

- $a_2 < b_2 < a_1 < b_1$:

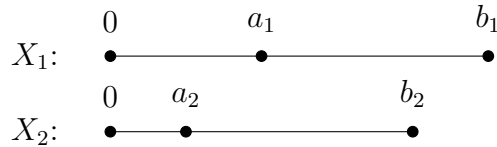


You simply ask low thresholds to both the variables. If X_1 has a positive response pick X_1 else pick X_2 .

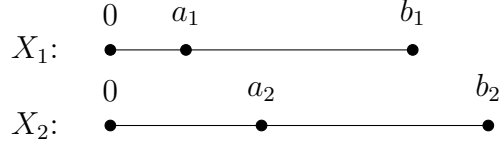
In all of the above four cases the reader can observe that we obtain the maximum value element, and hence the competitive ratio is 1 , and these are not the hard instances.

Only the following two are interesting cases :

- $a_2 < a_1 < b_2 < b_1$:



- $a_1 < a_2 < b_1 < b_2$:



We proceed with the first one of these cases.
 First we can write that

$$\mathbb{E}[OPT] = (1 - p_1 - q_1)(p_2 a_2 + b_2 q_2) + (p_1)(q_2 b_2 + (1 - q_2) a_1) + q_1 b_1$$

For the first item if we choose a high threshold, and obtain a negative response, then it is optimal to pick a low threshold for the second element. Let E_1 denote the expected value of the variable 1 after failing a high query. We get :

$$E_1 = \frac{p_1 a_1}{1 - q_1}$$

. We can further more have two cases : Case 1 : $a_2 < E_1$ and Case 2 : $a_2 > E_1$. The payoff obtained from these cases are as follows :

$$\text{Case 1.1 : } q_1 b_1 + (1 - q_1)(b_2 q_2 + (1 - q_2) E_1)$$

and

$$\text{Case 1.2 : } q_1 b_1 + (1 - q_1)(b_2 q_2 + a_2 p_2 + (1 - p_2 - q_2) E_1)$$

Similarly now suppose we pick the low threshold for the first item, and obtain a negative response, then it is optimal to just pick the second item. If we however receive a positive response, the expected value of the first item is :

$$E_2 = \frac{p_1 a_1 + b_1 q_1}{p_1 + q_1}$$

and hence this is the optimal threshold that we need to ask the second item. There are again two cases possible. Case 1 : $b_2 > E_2$, Case 2 : $b_2 < E_2$.

$$\text{Case 2.1 : } (1 - p_1 - q_1)(a_2 p_2 + b_2 q_2) + (p_1 + q_1)(b_2 q_2 + (1 - q_2) E_2)$$

and

$$\text{Case 2.2 : } (1 - p_1 - q_1)(a_2 p_2 + b_2 q_2) + E_2(p_1 + q_1)$$

Also note that Case 2.1 is strictly better than case 2.2 in terms of payoff. Depending on the

	High Threshold for X_1	Low Threshold for X_2
Case 1	$q_1 b_1 + (1 - q_1)(b_2 q_2 + (1 - q_2) E_1)$	$(1 - p_1 - q_1)(a_2 p_2 + b_2 q_2) + (p_1 + q_1)(b_2 q_2 + (1 - q_2) E_2)$
Case 2	$q_1 b_1 + (1 - q_1)(b_2 q_2 + a_2 p_2 + (1 - p_2 - q_2) E_1)$	$(1 - p_1 - q_1)(a_2 p_2 + b_2 q_2) + E_2(p_1 + q_1)$

instance, any of the cases can happen, and we pick the threshold that guarantees higher payoff. So

we need to minimise over the parameters, the maximum of the two payoffs.

Optimising for a_2 : If we keep $a_2 p_2$ fixed and vary p_2 notice that the payoff of OPT doesn't change. Among the algorithms, only the payoff of Case 1.2 changes, such that it increase when p_2 decreases, so for the worst case we want to increase p_2 we increase it till we reach $a_2 = E_1$.

Hence we obtain our first constraint that :

$$a_2 = \frac{p_1 a_1}{1 - q_1} \quad (12)$$

. And the common payoff of cases 1.1 and 1.2 is now :

$$q_1 b_1 + (1 - q_1)(b_2 q_2 + (1 - q_2)E_1)$$

Optimising for b_2 : If we keep $b_2 q_2$ fixed and increase q_2 by a unit amount, we obtain that OPT decreases by $a_1 p_1$. High threshold setting also reduces by $a_1 p_1$, while the low threshold setting case 1 reduces by $(p_1 + q_1)E_2 = a_1 p_1 + b_1 q_1$, and case 2 remains unaffected. So we keep on increasing q_2 till both the cases have the same payoff.

Similarly increasing q_2 only makes the CR worse as long as $b_2 > E_2$. So we increase q_2 till $b_2 = E_2$, beyond this increasing q_2 will not have an impact and at this point we get that the payoff obtained by the high threshold and the low threshold are the same.

We obtain the boundary condition when : $b_2 = E_2$ and payoff of both the high and low threshold match ie.

$$\begin{aligned} b_2 &= \frac{p_1 a_1 + b_1 q_1}{1 - p_1 - q_1} \quad (13) \\ q_1 b_1 + (1 - q_1)(b_2 q_2 + (1 - q_2)E_1) &= (1 - p_1 - q_1)(a_2 p_2 + b_2 q_2) + E_2(p_1 + q_1) \\ \implies q_1 b_1 + (1 - q_1)(b_2 q_2 + (1 - q_2)a_2) &= (1 - p_1 - q_1)(a_2 p_2 + b_2 q_2) + b_2(p_1 + q_1) \end{aligned}$$

Solving this we get

$$b_1 = a_1 + \frac{(p_1 + q_1)a_2 p_2 (1 - p_1 - q_1)}{q_1 p_1 q_2} \quad (14)$$

We can finally set $a_1 = 1$ as that only acts as a scaling factor.

So finally using 6.4 , 12 , 14 , 13 , we are left only to constrain over p_1, q_1, p_2, q_2 and find the worst case competitive ratio. We wrote a C++ program attached below that finds the worst case competitive ratio, along with the values at which it occurs.

We obtain that the worst case occurs at :

$$\begin{aligned} p_1 &= \frac{1}{2} \\ q_1 &= \epsilon \\ p_2 &= 2/3 \\ q_2 &= 1/3 \end{aligned} \quad (15)$$

where $\epsilon > 0$ is an arbitrarily small quantity. Plugging these in 6.4 , 12 , 14 , 13 we obtain that :

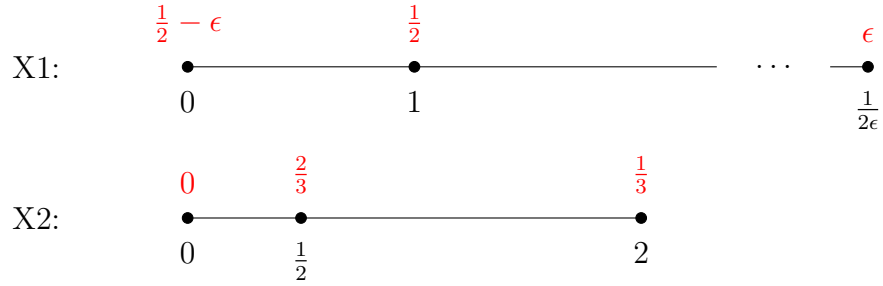
$$a_1 = 1 \tag{16}$$

$$b_1 = \frac{1}{2\epsilon} \tag{17}$$

$$a_2 = \frac{1}{2} \tag{18}$$

$$b_2 = 2 \tag{19}$$

The distribution looks as follows :



The values in red indicate the probability mass on each point.

If we calculate the payoff of the prophet it comes out to be $\frac{5}{3} - \epsilon$ and that of the algorithm it comes out to be $\frac{3}{2} + \epsilon$. Which gives us a competitive ratio upper bound of **9/10** The code used is the following :

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     double p1,q1,p2,q2,a1,b1,a2,b2;
6     a1 = 1;
7     double minima = 1;
8     double cr;
9     double min_ppqq[4] = {0.0,0.0,0.0,0.0};
10    int counter = 0;
11    for(q1 = 0 ; q1<0.01;q1+=0.0001){
12        for(p1 = 0 ; p1<=1-q1;p1+=0.0001){
13            a2 = (p1*a1)/(1-q1);
14            for(q2 = 0; q2<=1;q2+=0.0001){
15                for(p2 = 0 ; p2<=1-q2;p2+=0.0001){
16                    b1 = a1 + ((p1+q1)*a2*p2*(1-p1-q1))/(q1*p1*q2);
17                    b2 = (a1*p1+b1*q1)/(p1+q1);
18                    cr = ((1-p1-q1)*(a2*p2+b2*q2) +
19                        (a1*p1+b1*q1))/((1-p1-q1)*(a2*p2+b2*q2)+p1*(b2*q2+(1-q2)*a1)+b1*q1);
20                    if(cr<minima){
21                        minima = cr;
22                        min_ppqq[0]=p1;min_ppqq[1]=p2;min_ppqq[2]=q1;min_ppqq[3]=q2;
23                    }
24                }
25            }
26        }
27    }
28    cout<< "Part 1 results : "<<endl;
29    cout << "Minimum cr is " << minima << endl;
30    cout << "p1 = " << min_ppqq[0] << " p2 = " << min_ppqq[1] << " q1 = " <<
31        min_ppqq[2] << " q2 = " << min_ppqq[3] << endl;
32
33    counter = 0;
34
35    for(q1 = 0.01 ; q1<=1;q1+=0.01){
36        for(p1 = 0 ; p1<=1-q1;p1+=0.0001){
37            a2 = (p1*a1)/(1-q1);
38            for(q2 = 0; q2<=1;q2+=0.0001){
39                for(p2 = 0 ; p2<=1-q2;p2+=0.0001){
40                    b1 = a1 + ((p1+q1)*a2*p2*(1-p1-q1))/(q1*p1*q2);
41                    b2 = (a1*p1+b1*q1)/(p1+q1);
42                    cr = ((1-p1-q1)*(a2*p2+b2*q2) +
43                        (a1*p1+b1*q1))/((1-p1-q1)*(a2*p2+b2*q2)+p1*(b2*q2+(1-q2)*a1)+b1*q1);
44                    if(cr<minima){
45                        minima = cr;
46                        min_ppqq[0]=p1;min_ppqq[1]=p2;min_ppqq[2]=q1;min_ppqq[3]=q2;
47                    }
48                }
49            }
50        }
51    }

```

```

47         }
48     }
49 }
50
51 cout << "Minimum cr is " << minima << endl;
52 cout << "p1 = " << min_ppqq[0] << " p2 = " << min_ppqq[1] << " q1 = " <<
    min_ppqq[2] << " q2 = " << min_ppqq[3] << endl;
53 return 0;
54 }

```